EVENT VOLUNTEER HOUR LEDGER

College Presentation Slide Deck & Comprehensive Viva Voce Guide

Department: School of Sciences, CHRIST (Deemed to be University), Delhi NCR Campus
Domain Organization: AADIKALA & 6 Sub-Clubs (Dance, Emcee, Technical, Creative, Music, Hospitality)
Live Application: https://radhika2405-max.github.io/event-volunteer-hour-ledger/

Database Engine: MySQL 8.0 (InnoDB)	**Normal Form:** 3NF / BCNF Certified
Application Stack: React 18 (Vite, TS) + Flask REST	**Core Logic:** The Ledger Paradigm & ACID Locks

SLIDE 1: TITLE & SYSTEM INTRODUCTION

A Relational DBMS for University Service Hour Accounting & Certificate Governance

- **Institutional Context:** Developed for CHRIST University (School of Sciences) & AADIKALA.
- **Primary Challenge:** Coordinated volunteering across 6 sub-clubs (Dance, Emcee, Technical, Creative, Music, Hospitality).
- **Core DBMS Goal:** Replace untracked manual spreadsheets with an authoritative 3NF relational database.
- **Live Production Deployment:** Hosted on GitHub Pages with interactive dashboards, reports, and public verifier.

> ■■ ***Speaker Notes (What to say):***
> *Good morning respected professors and examiners. Today, we present our DBMS project: the Event Volunteer Hour Ledger. In university life, volunteer contributions across cultural, academic, and technical events represent hundreds of student labor hours. However, tracking these hours on spreadsheets or paper timesheets leads to lost updates, duplicate counting, and unverified certificates. We engineered an authoritative, 3NF-normalized relational database system that eliminates these anomalies, enforces strict ACID transactions, and provides verifiable certificate eligibility.*

SLIDE 2: THE REAL-WORLD PROBLEM: FRAGILITY OF SPREADSHEETS

Analyzing the 4 Fundamental Database Anomalies of Manual Timesheets

- **Mutable Aggregates & Lost Updates:** Directly editable total-hour cells overwrite previous values without a paper trail during concurrent edits.
- **Multi-Club Join Inflation:** Co-hosted events (e.g., SUNERGOS with 3 sub-clubs) multiply hours by 3 in unnormalized joins.
- **Premature Certification:** Students with pending unapproved hours claim certificates before faculty review.
- **Lack of Change Data Capture:** No audit log of who modified or approved an entry, why it was altered, or when.

> ■■ ***Speaker Notes (What to say):***
> *In our college organization, AADIKALA coordinates volunteers across multi-day fests like SUNERGOS and TARANG. Previous manual approaches failed because of fundamental database anomalies: first, mutable aggregates where total hours are directly editable without a paper trail; second, join inflation where multi-club events duplicate hours; third, students claiming certificates before faculty verify their hours; and fourth, a complete lack of change-data audit logging.*

SLIDE 3: THE ARCHITECTURAL SOLUTION: THE LEDGER PARADIGM

Decoupling Identity from Immutable Interval Entries & Dynamic SQL Views

- **The Ledger Principle:** Total hours are *NEVER* stored as a mutable column in the students table.
- **Formula:** Approved Hours = (SUM(verified_minutes) / 60.0) WHERE submission_status IN ('approved', 'corrected').
- **Strict 3-Tier Architecture:** React 18 + Vite Frontend -> Flask REST API -> MySQL 8.0 Database Engine.
- **Status Isolation:** Pending claimed hours and rejected submissions are strictly isolated from certified totals.

> ■■ ***Speaker Notes (What to say):***
> *To solve the mutable update anomaly, we adopted the Ledger Paradigm inspired by financial accounting. In our database, the students table does not have a total_hours column. Instead, every work interval is an immutable record in volunteer_hour_entries. Total hours are dynamically aggregated through authoritative database views. Only entries verified by faculty contribute to certified totals, strictly isolating pending and rejected submissions.*

SLIDE 4: RELATIONAL DATA MODELING & ER DESIGN

10 Normalized Relations Modeling Multi-Club Hierarchies & Associative Participation

- **Core Entities:** users (1:1 with students), clubs, events, certificate_policies.
- **Recursive Self-Join:** clubs.parent_club_id models parent AADIKALA and child sub-clubs.
- **Associative Junctions:** event_subclubs (M:N events to clubs), event_volunteers (M:N students to events).
- **Ledger & Auditing:** volunteer_hour_entries (1:N with participation), audit_logs (Change Data Capture).

> ■■ ***Speaker Notes (What to say):***
> *Here is our Entity-Relationship model. We separated authentication identity in users from academic records in students with a 1-to-1 relationship. We modeled the organizational hierarchy using a self-referential recursive foreign key in clubs, where parent_club_id links sub-clubs to AADIKALA. Events and sub-clubs share a Many-to-Many relationship resolved through event_subclubs, while student participation is captured through the associative entity event_volunteers.*

SLIDE 5: DATABASE NORMALIZATION JOURNEY (1NF TO 3NF & BCNF)

Systematic Decomposition Eliminating Insertion, Deletion, and Update Anomalies

- **1NF:** Atomic attributes enforced. Comma-separated sub-club lists decomposed into event_subclubs(event_id, club_id).
- **2NF:** Zero partial dependencies. In composite keys, non-key club data remains in clubs, event data in events.
- **3NF & BCNF:** Eliminated transitive dependencies. Program degree decoupled from login user; derived total hours removed.
- **Result:** Every determinant is a candidate key. No redundant facts stored across the entire schema.

> ■■ ***Speaker Notes (What to say):***
> *Our database strictly adheres to Third Normal Form. In 1NF, we eliminated repeating groups by breaking comma-separated sub-club lists into an associative junction table. In 2NF, we ensured zero partial functional dependencies on composite primary keys. In 3NF, we removed transitive dependencies—for instance, separating login credentials in users from degree programs in students, and removing any derived aggregate columns that would depend transitively on ledger entries.*

SLIDE 6: DATABASE INTEGRITY: CONSTRAINTS & REFERENTIAL ACTIONS

Engine-Enforced Rules Guaranteeing Mathematical & Temporal Realism

- **Check Constraints:** CHECK (end_datetime > start_datetime), CHECK (break_minutes >= 0), CHECK (claimed_minutes > 0).
- **Referential Integrity Actions:** ON DELETE CASCADE for participation child rows; ON DELETE RESTRICT on certificates.
- **Unique Key Constraints:** UNIQUE (event_id, student_id) prevents accidental duplicate event registrations.
- **Cryptographic Uniqueness:** UNIQUE (verification_token) protects public credential authenticity.

> ■■ ***Speaker Notes (What to say):***
> *Data integrity is enforced at the database engine level, not merely in the frontend. We implemented Check Constraints so that an event cannot end before it begins, and break minutes cannot be negative. We implemented composite unique constraints on (event_id, student_id) to prevent a volunteer from registering twice for the same event, and ON DELETE RESTRICT to protect issued certificates against accidental deletion.*

SLIDE 7: DATABASE VIEWS & SOLVING JOIN INFLATION

5 Specialized Virtual Relations Guaranteeing Non-Inflated Aggregation

- **vw_student_approved_hours:** Canonical student aggregation using SUM(CASE WHEN status='approved'...).
- **vw_certificate_eligibility:** Real-time evaluation against active policy (minimum 1,200 mins = 20 hrs).
- **vw_event_volunteer_summary:** Headcount and verified labor per event without duplicate club rows.
- **vw_subclub_contribution_summary:** Hierarchy-aware labor analytics for Dance, Emcee, Technical, Creative, Music, Hospitality.
- **vw_pending_review_queue:** Actionable queue for faculty approvers without exposing sensitive student fields.

> ■■ ***Speaker Notes (What to say):***
> *One of our proudest database achievements is solving Join Inflation. When an event has multiple collaborating sub-clubs, a naive multi-table join multiplies hours by the number of sub-clubs. We designed five specialized SQL views. In vw_student_approved_hours, hours attach to the volunteer participation ID, and we use conditional aggregation: SUM(CASE WHEN submission_status = 'approved' THEN verified_minutes ELSE 0 END). This guarantees mathematical precision across all reports.*

SLIDE 8: DATABASE PROGRAMMABILITY: TRIGGERS & CHANGE DATA CAPTURE

Event-Driven Business Rules & Automated Forensic Audit Logging

- **trg_vhe_before_insert:** Validates non-negative breaks and positive claimed minutes before record creation.
- **trg_vhe_before_update:** Enforces state-machine rules. If status='rejected' without reason, raises SQLSTATE '45000'.
- **trg_vhe_after_update:** Change Data Capture (CDC)—captures before/after JSON states into audit_logs.
- **Forensic Accountability:** Every hour approval, modification, or rejection is permanently timestamped with actor ID.

> ■■ ***Speaker Notes (What to say):***
> *We utilized advanced database programmability. Our BEFORE UPDATE trigger on hour entries enforces state transition integrity: if a faculty member attempts to reject hours without providing a written reason, the database raises an exception and aborts the statement. Meanwhile, the AFTER UPDATE trigger automatically records before and after JSON snapshots in audit_logs, providing forensic traceability.*

SLIDE 9: STORED PROCEDURES & ACID CONCURRENCY CONTROL

Row-Level Locking Preventing Double Issuance & Lost Updates

- **sp_calculate_student_hours:** Encapsulated calculation returning approved, pending, rejected minutes.
- **sp_refresh_certificate_eligibility:** Dynamic evaluation against policy threshold.
- **sp_issue_certificate (ACID Transaction):**
- - START TRANSACTION with row-level locks: SELECT ... FOR UPDATE on candidate student.
- - Verifies approved minutes >= 1,200 mins. If below threshold -> ROLLBACK.
- - Verifies no active certificate issued. If already issued -> ROLLBACK.
- - Generates immutable snapshot and SHA-256 verification token, then commits.

```
START TRANSACTION;
SELECT * FROM certificate_policies WHERE policy_id = 1 FOR SHARE;
SELECT * FROM students WHERE student_id = 4 FOR UPDATE;
-- If approved_minutes < threshold THEN ROLLBACK; END IF;
INSERT INTO certificates (...) VALUES (...);
COMMIT;
```

■■ ***Speaker Notes (What to say):***
Certificate issuance requires strict ACID compliance. If a student is near the 20-hour threshold, simultaneous approval requests could trigger race conditions. In our stored procedure sp_issue_certificate, we execute START TRANSACTION and obtain row-level exclusive locks with SELECT ... FOR UPDATE. If the student has already received a certificate, or their approved minutes drop below the threshold, the transaction rolls back cleanly with custom SQL error codes.

SLIDE 10: QUERY OPTIMIZATION & BALANCED B-TREE INDEXING STRATEGY

Sub-Millisecond Execution Times on High-Volume Reporting Queries

- **Clustered Primary Key Indexes:** O(log N) row lookups across all 10 relational tables.
- **Secondary B-Tree Indexes:** idx_users_email, idx_students_reg_no, idx_cert_verification_token.
- **Composite B-Tree Indexes:** idx_vhe_part_status on (participation_id, submission_status) for rapid view joins.
- **Execution Plan (EXPLAIN):** Eliminates table scans (ALL), executing queries via index seeks (ref/range) in < 50ms.

■■ ***Speaker Notes (What to say):***
To ensure scalability across thousands of volunteers and entries, we engineered a dedicated B-tree indexing strategy. Beyond primary keys, we created secondary indexes on lookup columns like registration numbers and tokens, and composite indexes on (participation_id, submission_status). Using EXPLAIN, we verified that our reporting queries execute using efficient index range lookups under 50 milliseconds rather than full table scans.

SLIDE 11: LIVE SYSTEM FEATURES & CERTIFICATE VERIFICATION PORTAL

React + Vite Frontend Hosted Live on GitHub Pages

- **Live Application URL:** https://radhika2405-max.github.io/event-volunteer-hour-ledger/
- **Volunteer Workspace:** Personal hours breakdown, event registrations, pending submission status.
- **Faculty Approver Hub:** Live queue with one-click approval, corrections, and mandatory rejection notes.
- **Eligibility Report Engine:** Search, multi-criteria filtering (Year, Programme, Hours) and instant CSV export.
- **Public Verification Portal:** Unauthenticated SHA-256 token validator verifying student certificate authenticity.

> ■■ ***Speaker Notes (What to say):***
> *Our user interface is deployed live on GitHub Pages. It features five specialized workspaces: the Volunteer Portal showing personal hours, the Approver Hub where faculty review submissions, the Certificate Eligibility engine tracking the 20-hour requirement, our search-and-filter report table with CSV export, and a public verification screen where anyone can enter a token to verify a certificate's authenticity.*

SLIDE 12: CONCLUSION & ACADEMIC PROJECT ACHIEVEMENTS

A Production-Ready Demonstration of Comprehensive DBMS Principles

- **Summary of Accomplishments:** Replaced manual spreadsheets with an enterprise 3NF MySQL database.
- **DBMS Theory Realized:** Normalization, Constraints, Views, Triggers, Stored Procedures, ACID, B-Tree Indexes.
- **Real-World Domain Impact:** Solved Join Inflation and eliminated lost updates for CHRIST University's AADIKALA.
- **Viva Defense Preparedness:** Ready for faculty examination and interactive query execution.

> ■■ ***Speaker Notes (What to say):***
> *In conclusion, this project demonstrates how foundational DBMS theory—normalization, relational algebra, constraints, ACID transactions, and indexing—directly solves real-world campus administrative challenges. Thank you, and we are now ready for your questions and the live demonstration.*

PART 2: FACULTY VIVA VOCE DEFENSE QUESTIONS & ANSWERS

Strategic Answers Prepared for External Examiners & Evaluation Panel

Q1: Why did you use the 'Ledger Paradigm' instead of adding a 'total_hours' column in students?
Adding total_hours to students introduces a Transitive Dependency (StudentID -> Entries -> TotalHours), violating 3NF. Furthermore, it creates a severe Update Anomaly and Race Condition: if two coordinators approve different sessions simultaneously, concurrent updates overwrite each other, causing lost hours. In the Ledger Paradigm, every entry is an immutable record, and total hours are dynamically aggregated via SQL views, guaranteeing 100% mathematical consistency.

Q2: What is Join Inflation and how did your schema prevent it?
Join Inflation occurs when joining across a Many-to-Many relationship. If Event A has 3 sub-clubs in event_subclubs, joining events directly to event_subclubs and then to volunteer_hour_entries multiplies every hour entry by 3. We solved this by attaching hour entries strictly to event_volunteers(participation_id). Club associations remain in the separate junction table and are aggregated independently with COUNT(DISTINCT).

Q3: How does your database handle ACID transactions during certificate issuance?
In stored procedure sp_issue_certificate, we wrap issuance inside START TRANSACTION ... COMMIT. We use row-level exclusive locks using SELECT ... FOR UPDATE on the student record and FOR SHARE on the policy. If the student's verified hours are below the 1,200-minute threshold, or if an active certificate already exists, the procedure calls ROLLBACK and raises an exception, guaranteeing Atomicity and Isolation.

Q4: What is the difference between a WHERE clause and a HAVING clause in your eligibility query?
In reports.sql Query 1, the WHERE clause filters individual records before grouping—we use WHERE submission_status IN ('approved', 'corrected') so pending or rejected timesheets are excluded immediately. The HAVING clause filters groups after aggregation—we use HAVING SUM(verified_minutes) >= 1200 to select only those students whose cumulative sum reaches or exceeds the 20-hour policy threshold.

Q5: What triggers did you implement and what business rules do they enforce?
We implemented three triggers in triggers.sql. Our BEFORE INSERT trigger ensures positive claimed durations and non-negative breaks. Our BEFORE UPDATE trigger acts as a state-machine gatekeeper: if a reviewer marks an entry as 'rejected' without providing a rejection_reason, it raises SQL error 45000 and aborts the update. Finally, our AFTER UPDATE trigger captures before and after states as JSON objects and automatically records them into audit_logs.

Q6: How do your B-Tree indexes improve query performance?
Without indexes, finding a student by registration number or calculating hours requires a full table scan O(N). We created clustered indexes on primary keys, secondary B-tree indexes on lookup columns (university_registration_number, email, verification_token), and composite B-tree indexes on (participation_id, submission_status). EXPLAIN shows MySQL performs index range lookups in O(log N) time (< 50ms).